

PRISM v0.17 — Protocol Specification

Vision

Prism is a blockchain protocol designed around three equally important forms of contribution:

Proof of Stake secures.

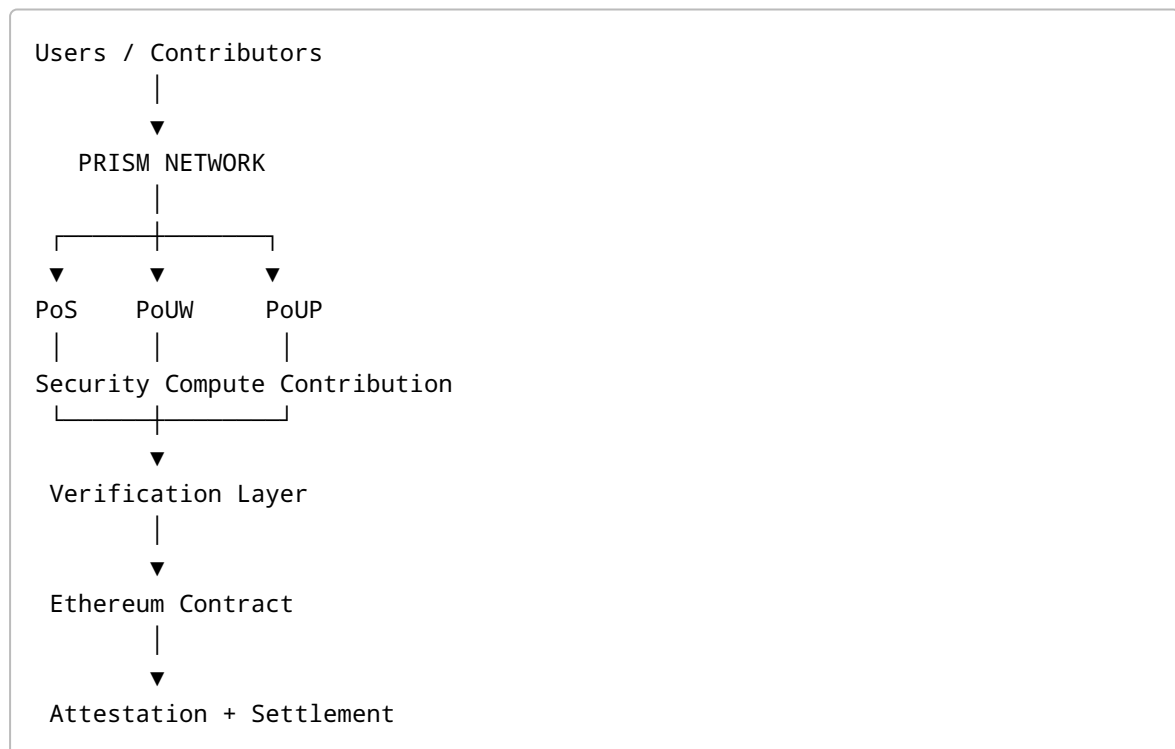
Proof of Useful Work computes.

Proof of Useful Participation grows.

Prism aims to reward not only capital, but also useful computation and measurable contributions to the network.

Core Architecture

Prism v0.17 combines an off-chain compute and participation layer with Ethereum as a settlement and attestation layer.



Ethereum does not execute heavy computation. Prism performs and verifies useful work off-chain, while Ethereum records proofs, hashes, identities and settlement events.

Tokenomics

Token: PRISM

Maximum Supply: 100,000,000 PRISM

Supply: Permanently capped.

Allocation

- 20,000,000 PRISM — Proof of Stake
- 20,000,000 PRISM — Proof of Useful Work
- 20,000,000 PRISM — Proof of Useful Participation
- 15,000,000 PRISM — Ecosystem & Grants
- 10,000,000 PRISM — Treasury
- 10,000,000 PRISM — Team
- 5,000,000 PRISM — Launch & Liquidity

60% of the total supply can only be earned by securing, computing for, or contributing to the network.

Network rewards are distributed through epoch-based emission and decrease over time. No tokens can be created beyond the 100 million maximum supply.

Proof of Stake — PoS

PoS provides the security foundation of Prism.

Validators stake PRISM and receive rewards based on factors including:

Stake
×
Validator uptime
×
Performance

Prism may use non-linear stake weighting to reduce excessive concentration of economic power while preserving economic security.

Validators can participate in the verification of PoUW jobs and PoUP contributions.

Proof of Useful Work — PoUW

PoUW rewards verified computation that performs useful work.

Prism v0.17 Reference Workload

Matrix multiplication:

$$C = A \times B \bmod p$$

The worker performs the expensive computation.

Prism verifies the result using probabilistic verification such as Freivalds' algorithm:

$$A \times (B \times r) == C \times r$$

The goal is:

Computation → expensive
Verification → inexpensive

Job Lifecycle

```
OPEN
↓
CLAIMED
↓
COMPUTED
↓
VERIFYING
↓
VERIFIED
↓
REWARDED
```

Invalid work receives no PoUW reward.

PoUW rewards depend on factors such as:

```
Computational difficulty
×
Verified work
×
Worker reliability
```

Ethereum stores the job hash, result hash, worker identity and verification receipt rather than the full computation.

Proof of Useful Participation — PoUP

PoUP rewards contributions that measurably improve Prism.

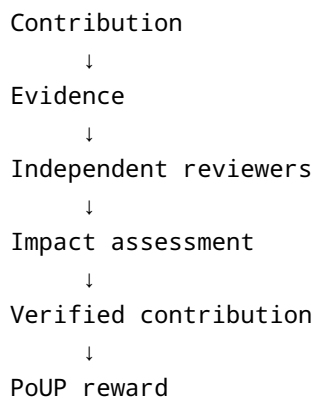
Eligible contributions may include:

- protocol code
- security disclosures
- documentation
- research
- infrastructure
- integrations
- governance work

Social activity such as likes, followers or message volume does not qualify as useful participation.

Verification

A contribution includes cryptographic or externally verifiable evidence.



A contributor cannot validate their own work.

The initial model uses multiple independent reviewers and a minimum approval threshold.

Anti-Sybil Principles

PoUP incorporates mechanisms designed to make fake identities and contribution farming economically ineffective:

- no self-validation
- contribution evidence
- reviewer diversity
- reputation
- diminishing returns for excessive submissions
- penalties for fraudulent reviewing
- optional submission bonds
- future collusion detection

A conceptual contribution score is:

```
PoUP Score =  
Base Value  
× Quality  
× Impact  
× Reputation  
× Anti-Sybil Factor
```

PoUP rewards are distributed proportionally from the current epoch's PoUP reward pool.

Reward Model

Each epoch creates three separate reward budgets:

```
PoS Pool  
PoUW Pool  
PoUP Pool
```

Each mechanism distributes its own pool proportionally according to verified contribution.

This preserves the economic principle that the three pillars receive equal long-term token allocations while allowing each proof mechanism to use its own scoring system.

Prism v0.17 Hackathon Demo

The target demo follows one complete useful-work lifecycle:

1. User connects an Ethereum wallet
2. User submits a useful computation job
3. Prism worker claims the job
4. Worker computes the result
5. Prism validators verify the work
6. Job becomes VERIFIED
7. Ethereum receives the attestation
8. Worker receives a PRISM reward
9. Dashboard displays the completed job

The dashboard should expose:

- block height
- validators
- active jobs
- completed useful work
- PoS / PoUW / PoUP activity
- worker addresses

- verification status
 - rewards
 - Ethereum transaction references
-

Design Principle

Prism is not designed around the idea that capital alone defines contribution.

Its economic model recognizes three forms of value:

Capital	→ secures the network
Computation	→ performs useful work
Participation	→ improves the network

Protocol Thesis

Prism turns decentralized consensus into a system that rewards security, computation and contribution equally.